

AI Development Prompt – Preferential Rules of Origin Calculation System

You are a Senior Solution Architect and Principal Software Engineer.

Build an enterprise-grade application for **Preferential Rules of Origin Calculation** based on EU Trade Agreements.

The objective is to determine whether a manufactured product qualifies as an **Originating Product** according to preferential rules of origin.

The software should be designed so that new agreements, origin rules and product rules can easily be added without changing business logic.

Do NOT create a demo project. Build a production-ready solution following enterprise software engineering best practices.

Technology Stack

Frontend

- React 19
 - TypeScript
 - Material UI
 - React Router
 - React Hook Form
 - Zod Validation
 - Axios
 - TanStack Query (React Query)
 - Redux Toolkit (only if necessary)
 - Responsive Design
 - Dark/Light Theme
-

Backend

- ASP.NET Core 9 Web API
- C#
- Clean Architecture
- CQRS
- MediatR

- FluentValidation
- AutoMapper
- Dependency Injection
- Entity Framework Core
- Dapper
- PostgreSQL
- Serilog
- Swagger/OpenAPI
- Global Exception Middleware
- Health Checks

Database

PostgreSQL

Architecture

Follow Clean Architecture strictly.

Solution Structure

/src

```
Api
Application
Domain
Infrastructure
Persistence
Shared
Tests
```

Never allow API to access Infrastructure directly.

Only Application should contain CQRS.

Data Access Rules

Read Operations

ALL SELECT queries must use Dapper.

Examples

- Search Product
- Search Rule
- Search Agreement
- Search Country
- Search HS Codes
- Search Calculations
- Dashboard

must use Dapper.

Write Operations

ALL

Insert

Update

Delete

must use Entity Framework Core.

Repositories should only be used for write operations.

CQRS

Every feature must contain

Command

Command Handler

Query

Query Handler

Validator

DTO

Response Model

Example

CreateProductCommand

CreateProductCommandHandler

GetProductByIdQuery

GetProductByIdQueryHandler

Logging

Implement enterprise logging.

Use Serilog.

Log

Errors

Warnings

Information

Performance

Unhandled Exceptions

Validation Errors

Database Errors

Request Execution Time

Log to

Console

Rolling File

PostgreSQL (optional)

Every exception should contain

Timestamp

User

API

Stack Trace

Correlation Id

Request Body

Response Code

Error Handling

Create Global Exception Middleware.

Never return stack traces to client.

Return consistent error response.

Example

```
{ "success": false, "message": "...", "errors": [], "traceId": "" }
```

Dependency Injection

Register every service properly.

Handlers

Repositories

Validators

AutoMapper

DbContext

Dapper Connection

Logging

Configuration

Authentication

Everything must use constructor injection.

No service locator.

Authentication

Implement JWT Authentication.

Include

Login

Refresh Token

Role Based Authorization

Admin

Operator

Viewer

Functional Modules

1. Trade Agreements

CRUD

Fields

Agreement Name

Country

Effective Date

Expiry Date

Status

Description

2. Countries

CRUD

Country Name

ISO Code

Region

Is EU Member

3. HS Codes

CRUD

HS Code

Description

Chapter

Heading

Sub Heading

4. Product Rules

Each HS Code can have multiple rules.

Rule Types

Value Added Rule

Change of Tariff Classification

Specific Processing

Alternative Rule

Tolerance Rule

Cumulation Rule

Origin Quota

Each rule should have

Rule Description

Threshold

Percentage

Operation

Applicable Countries

Effective Date

Priority

Version

5. Materials

CRUD

Material Name

Origin Country

HS Code

Supplier

Originating

Non-Originating

Cost

Quantity

Currency

6. Finished Products

CRUD

Product Name

HS Code

Ex Works Price

Manufacturer

Country

Materials Used

Manufacturing Process

7. Origin Calculation

This is the core module.

Input

Finished Product

Materials

Agreement

Country

HS Code

Manufacturing Operations

The engine should determine

Originating

Non-Originating

Why

Applicable Rules

Calculation Details

Decision

Pass

Fail

Result Explanation

8. Rule Engine

Design using Strategy Pattern.

Each rule should be implemented independently.

Example

IOOriginRule

ValueAddedRule

TariffShiftRule

SpecificOperationRule

ToleranceRule

CumulationRule

OriginQuotaRule

The engine should evaluate dynamically.

Future rules should be pluggable.

Business Logic

Implement calculations for

Wholly Obtained

Value Added

Maximum Non-Originating %

Change of Tariff Heading

Change of Tariff Subheading

Specific Manufacturing Operations

Tolerance

Cumulation

Regional Cumulation

Diagonal Cumulation

Full Cumulation

Origin Quotas

Support multiple agreements.

Calculation Result

Return

Decision

Reason

Applied Rules

Rejected Rules

Percentages

Cost Breakdown

Rule Evaluation Tree

UI

Create professional UI.

Modules

Dashboard

Countries

Trade Agreements

HS Codes

Rules

Products

Materials

Origin Calculator

Reports

Settings

Use

Material UI DataGrid

Dialogs

Autocomplete

Pagination

Filtering

Sorting

Export to Excel

Export PDF

Reports

Calculation History

Agreement Report

HS Code Report

Country Report

Rule Usage Report

Export

Excel

PDF

CSV

Audit Trail

Track

Created By

Updated By

Deleted By

Created Date

Modified Date

IP Address

Machine

Validation

Use FluentValidation.

Client Validation

Server Validation

No duplicated validation logic.

Testing

Create

Unit Tests

Integration Tests

Repository Tests

Calculation Engine Tests

CQRS Tests

Coverage should exceed 80%.

Coding Standards

Use SOLID.

Use DRY.

Use KISS.

Use Repository Pattern only for write operations.

Never use repositories for SELECT.

Always use Dapper for queries.

No business logic inside controllers.

No SQL inside controllers.

No static helper classes.

No magic strings.

Use constants.

Use extension methods where appropriate.

Use async/await everywhere.

Support cancellation tokens.

Performance

Pagination

Caching where appropriate

Indexes

Optimized SQL

Connection Pooling

Avoid N+1 queries

Batch operations

Deliverables

Generate

1. Complete Solution Structure
2. Database Schema
3. Entity Relationship Diagram
4. PostgreSQL Scripts
5. Entity Framework Migrations
6. Clean Architecture Project
7. Backend API
8. React Frontend
9. Complete Rule Engine
10. CQRS Implementation
11. Dapper Queries
12. EF Core Commands
13. Authentication
14. Logging
15. Exception Handling
16. Swagger
17. Unit Tests

- 18. Integration Tests
 - 19. Docker Support
 - 20. Docker Compose
 - 21. README
 - 22. API Documentation
 - 23. Architecture Documentation
 - 24. Sequence Diagrams
 - 25. Class Diagrams
 - 26. Deployment Guide
 - 27. Sample Seed Data
-

Development Instructions

Do NOT generate everything at once.

Develop feature-by-feature.

For each feature:

- 1. Database
- 2. Domain
- 3. Application
- 4. Infrastructure
- 5. API
- 6. React UI
- 7. Unit Tests
- 8. Integration Tests

Wait for confirmation before moving to the next module.

Ensure every generated module compiles successfully before proceeding.

Maintain production-quality code throughout the project.